

PREGUNTAS

Terminal inteligente

1. Muestra dos soluciones que permitan transformar la cadena ingenioso en ingeniero.
Indique el costo de cada solución.
 $a = 1, d = 2, r = 3, i = 2, k = 1.$

Operaciones	cadena
Avanzar i	Ingenioso
Avanzar n	Ingenioso
Avanzar g	Ingenioso
Avanzar e	Ingenioso
Avanzar n	Ingenioso
Avanzar i	Ingenioso
Insertar e	Ingenieoso
Insertar r	Ingenieroso
Avanzar o	Ingenieroso
Kill	Ingeniero

Costo 12

Operaciones	cadena
Avanzar i	Ingenioso
Avanzar n	Ingenioso
Avanzar g	Ingenioso
Avanzar e	Ingenioso
Avanzar n	Ingenioso
Avanzar i	Ingenioso
Reemplazar con e	Ingenieso

Reemplazar con r	Ingeniero
Avanzar o	Ingeniero

Costo 12+1

2. Muestra dos soluciones que permitan transformar la cadena francesa en ancestro. Indique el costo de cada solución.

Operaciones	cadena
Borrar f	rancesa
Borrar r	ancesa
Avanzar a	ancesa
Avanzar n	ancesa
Avanzar c	ancesa
Avanzar e	ancesa
Avanzar s	ancesa
Reemplazar con t	ancest
Insertar r	ancestr
Insertar o	Ancestro

Costo: 16

Operaciones	cadena
Insertar a	afrancesa
Insertar n	anfrancesa
Insertar c	ancfrancesa
Insertar e	ancefrancesa
Insertar s	ancesfrancesa
Insertar t	ancestfrancesa
Insertar r	ancestrfrancesa

Insertar o	ancestrofrancesa
kill	Ancestro

Costo 17

- Sea $x[1..n]$ la cadena a transformar en $y[1..n]$, caracterice la estructura de una solución óptima, esto es, indique como la solución óptima está compuesta de otras soluciones óptimas
R/ la solución óptima se caracteriza de subproblemas más pequeños como por ejemplo: queremos transformar gato en perro, si lo dividimos tenemos varios subproblemas, transformar **ga** en **pe** y transformar **gat** en **per**, si notamos se repite el subproblema de transformar **ga** en **pe**. Así que, almacenaremos los subproblemas en una matriz $m \times n$, donde m sea el tamaño de la cadena inicial, y en el tamaño de la cadena objetivo o final.

- Explique cuál es la forma correcta de completar la matriz M . Indique en qué posición de la matriz quedará la solución al problema original.

Nota: usaremos estos costos para el ejemplo $a = 1$, $d = 2$, $r = 3$, $i = 2$, $k = 1$

La forma en que llenaremos la matriz, será desde la última posición m, n hasta la $0,0$. Y La solución óptima se verá alojada en la posición $0,0$

El ciclo es el siguiente:

```
for cursor1 in range(m, -1, -1):
    for cursor2 in range(n, -1, -1):
```

Donde m es el tamaño de la palabra 1 o inicial y n es el tamaño de la palabra 2 u objetivo

```
[(0, 0), (0, 1), (0, 2), (0, 3)]
[(1, 0), (1, 1), (1, 2), (1, 3)]
[(2, 0), (2, 1), (2, 2), (2, 3)]
[(3, 0), (3, 1), (3, 2), (3, 3)]
```

Utilizaremos esta matriz 4×4 para explicar la lógica que usamos

Al empezar en la posición $3,3$, lo que interpretamos es ¿qué hace falta hacer para completar correctamente la transformación de la cadena? La posición m,n es 0 porque no falta nada o ambas cadenas están vacías siendo $0,0$

De ahí en adelante nos preguntamos lo mismo, ¿qué hace falta para terminar de transformar la cadena? ¿Qué debemos hacer para transformar lo que queda en esta posición más las anteriores exploradas?

A continuación tenemos el algoritmo que rellena la matriz en casos que no son base

$matriz[cursor1 + 1][cursor2 + 1] + a < matriz[cursor1][cursor2]$,

para la función **avanzar** usamos +1 en ambos cursores porque para llegar al estado actual ambos cursores debieron avanzar 1 y a esa posición se le añade el costo de avanzar, así que estamos comparando con la posición posterior en la matriz que sería la anterior.

matriz[cursor1 + 1][cursor2 + 1] + r < matriz[cursor1][cursor2]:

en **reemplazar**, usamos la misma lógica de avanzar, ya que cambia la letra de la palabra inicial y luego el cursor avanza

matriz[cursor1 + 1][cursor2] + d < matriz[cursor1][cursor2]:

En **eliminar** usamos solamente cursor1+1, porque el subproblema actual al eliminar un elemento es lo mismo que el subproblema cuando los cursores están (cursor1+1,cursor2) solamente que agregando el costo de eliminar +d. El +1 lo que quiere decir es que devolvimos el cursor1 una posición, creando el estado de las palabras como si se hubiera eliminado un elemento, lo que quiere decir que es como si faltara una letra menos en palabra1 por procesar.

matriz[cursor1][cursor2 + 1] + i < matriz[cursor1][cursor2]:

en la función **insertar**, cursor2 se le suma uno porque al insertar un elemento nos faltan menos letras de la palabra 2 por procesar

k + (tamaño_palabra2 - cursor2) * i < matriz[cursor1][cursor2]:

en la función **kill** se tiene en cuenta si cualquiera de las funciones que se consideraron en este punto, es peor que borrar todo lo que queda e insertar lo que queda de la palabra2

Al final, después de ir en reversa en ambas palabras, tenemos que para procesar las letras restantes que en la posición 0,0 serían todas, es igual a lo que el algoritmo de comparación aloje en la matriz en esta posición.

[5, 5, 3, 1]

[6, 4, 3, 1]

[7, 5, 3, 1]

[6, 4, 2, 0]

Esta es la matriz resultante de transformar cat en car :

-Las palabras también se toman en reversa, así que la posición (2,1) significa que estamos procesando la última letra de cat y las dos últimas letras de car

-En la posición (2,1) tenemos la siguiente situación:

- transformar t en ar = matriz[cursor1+1][cursor2+1] + r = **3 + 2 = 5**

Porque el algoritmo tomó reemplazar y no otro:

matriz[cursor1+1][cursor2+1] + a, no puedo avanzar porque t es distinto de a

matriz[cursor1+1][cursor2+1] + r, 2 + 3 = 5 se asigna porque es menor a infinito

matriz[cursor1+1][cursor2] + d, 4 + 2 = 6 simplemente dio un número mayor

matriz[cursor1][cursor2+1] + i, 3 + 2 = 5 tiene que ser estrictamente menor que el número en la

posición actual que sería 5

$k+(n-\text{cursor2})*i = 1 + (3-1)*2 = 5$ no es estrictamente menor a 5

5. Utilice la recurrencia. Explique con ejemplos cada uno de los valores que calcula para la matriz M. Puede suponer los siguientes costos para las operaciones: $a = 1$, $d = 2$, $r = 3$, $i = 2$, $k = 1$.

R/

m tamaño palabra inicial

n tamaño palabra final u objetivo

Casos base:

matriz[cursor1][cursor2] = 0 transformar lo que falta en dos cadenas procesadas o vacías cero.

matriz[cursor1][n] = min((m-cursor1)*d, k) el costo de transformar cualquier cadena en una ya procesada o vacía es el mínimo de borrar o hacer kill en lo que sobra de la palabra 1. Si la palabra es **gatos** y se quiere transformar en **gato**, al estar en el tamaño 1 o solo queda borrar o hacer kill.

O si queremos transformar gatos en cadena vacía ($n=0$) solo se puede borrar o hacer kill

matriz[m][cursor2] = ((n-cursor2) * i) el costo de transformar una cadena vacía o procesada en una cadena cursor2 es el costo de insertar lo que hace falta de cadena n

Relación de recurrencia para m[i][j] cuando $i > 0$ y $j > 0$:

Tengamos en cuenta las siguientes operaciones:

Si $\text{cursor1} < m$ y $\text{cursor2} < n$

$\text{Matriz}[\text{cursor1}][\text{cursor2}] = \min(\text{matriz}[\text{cursor1}+1][\text{cursor2}+1] + a, \\ \text{matriz}[\text{cursor1}+1][\text{cursor2}+1] + r, \\ \text{matriz}[\text{cursor1}+1][\text{cursor2}] + d, \\ \text{matriz}[\text{cursor1}][\text{cursor2}+1] + i, \\ k+(n-\text{cursor2})*i)$

Ejemplo

$a = 1$, $d = 2$, $r = 3$, $i = 2$, $k = 1$.

Transformar cat en car

Casos base:

[inf, inf, inf, 1]

[inf, inf, inf, 1]

[inf, inf, inf, 1]

[6, 4, 2, 0]

Transformar cadena vacía a algo:

- " " en " " = 0

- " " en r = 2 costo insertar

- " " en ar = 4

" " en car = 6

Transformar algo en cadena vacía:

- t en " " = 1 costo kill

- at en " " = 1

- cat en “ “ = 1

Si cursor1 < m y cursor2 < n

[inf, inf, inf, 1]

[inf, inf, inf, 1]

[7, 5, 3, 1]

[6, 4, 2, 0]

- t en r = matriz[cursor1+1][cursor2+1] + r = 0 + 3 = 3

- t en ar = matriz[cursor1+1][cursor2+1] + r = 3 + 2 = 5

- t en car = matriz[cursor1+1][cursor2+1] + r = 4 + 3 = 7

[inf, inf, inf, 1]

[6, 4, 3, 1]

[7, 5, 3, 1]

[6, 4, 2, 0]

- at en r = matriz[cursor1][cursor2+1] + i, = 1 + 2 = 3

- at en ar = matriz[cursor1+1][cursor2+1] + r, = 3 + 1 = 4

- at en car = matriz[cursor1][cursor2+1] + i, = 4 + 2 = 6

[5, 5, 3, 1]

[6, 4, 3, 1]

[7, 5, 3, 1]

[6, 4, 2, 0]

- cat en r = matriz[cursor1][cursor2+1] + i, = 1 + 2 = 3

- cat en ar = matriz[cursor1][cursor2+1] + i, = 3 + 1 = 4

- cat en car = matriz[cursor1+1][cursor2+1] + a, , = 4 + 1 = 5 **Solucion Optima**

6. Complejidad de las 3 soluciones

m y n son las longitudes de palabra 1 y palabra 2

1. Estrategia por fuerza bruta (solución ingenua)

El tiempo computacional es $O(4^{(\max(n, m))})$. El 4 es por la cantidad de llamados recursivos que serían inserción, borrado, reemplazo, y avanzar (si los caracteres coinciden). Kill no se incluye porque es una asignación y no un llamado recursivo.

El espacio computacional es $O(\max(n, m))$

2. Estrategia de programación dinámica

El tiempo computacional es $O(n \times m)$, ya que sería el tamaño de la matriz .

El espacio computacional es $O(n \times m)$

3. Estrategia de programación voraz

El tiempo computacional es $O(n+m)$, El algoritmo recorre palabra1 y palabra2 de principio a fin, procesando cada carácter solo una vez en un bucle while que termina cuando ambos cursores llegan al final de sus respectivas palabras.

El espacio computacional es $O(n+m)$.

Subasta

1. Muestre dos asignaciones de las acciones para $A = 1000$, $B = 100$, $n = 2$, la oferta $< 500, 100, 600 >$, la oferta $< 450, 400, 800 >$ y la oferta del gobierno $< 100, 0, 1000 >$. Indique el valor v_r para la solución

R/ dos asignaciones pueden ser:

- Ofertante 1: 600, ofertante 2: 400, gobierno: 0. Valor v_r : $300000 + 180000 = 480000$
- Ofertante 1: 500, ofertante 2: 400, gobierno 100. Valor $v_r = 250000 + 180000 + 10000 = 440000$

2. Considere el algoritmo que lista las posibles asignaciones:

$$< y_1, y_2, \dots, \sum_{i=1}^{n-1} y_i = A >$$

Donde $y_i \in \{0, M_i\}$, luego calcula el v_r para cada asignación y selecciona la mejor.

Utilice el algoritmo anterior para la entrada $A = 1000$, $B = 100$, $n = 4$, $< 500, 400, 600 >$, $< 450, 100, 400 >$, $< 400, 100, 400 >$, $< 200, 50, 200 >$, la oferta del gobierno $< 100, 0, 1000 >$.

Indique si el algoritmo anterior siempre encuentra la solución óptima

R/ siempre encuentra la solución óptima, ya que hace la combinación de asignar a cada oferente entre 0 y el número máximo de acciones que puede comprar dicho ofertante (M_i), para cada combinación de asignaciones calcula cuánto valor recibido se obtiene y nos quedamos con la mejor.

3. Caracterice la estructura de una solución óptima, esto es, indique la forma de los subproblemas y como la solución óptima la componen.

R/ La solución óptima a este problema se compone de subproblemas con base en la asignación de acciones, es decir, si voy a asignar n acciones, la solución óptima se compone de asignar $n - k$ acciones, donde k es la cantidad de acciones que trato de asignar a x_i oferente. En otras palabras, si conozco el valor óptimo de asignar $n - k$ acciones, basándonos en ese subproblema, puedo lograr determinar el valor máximo para vender n acciones.

Si yo sé el resultado al problema de asignar 5,4,3,2,1,0 acciones entre **2 oferentes**, puedo determinar cual es el valor máximo, de asignar cualquier cantidad de acciones (menor o igual que 5) entre **3 oferentes**. En otras palabras, si el problema completo es asignar 5 acciones entre 3 oferentes y tengo almacenados la mejor forma de asignar las acciones entre 2 oferentes puedo

reutilizar los subproblemas, de tal forma, que al intentar asignar 1 acción al oferente 3 puedo tomar el subproblema en donde asigno 4 acciones entre los dos primeros oferentes y le sumo lo que paga el oferente 3 por la acción asignada, si es mayor, acabo de encontrar una mejor solución, evitando recalcular la forma de asignar 4 acciones entre los 3 oferentes

4. Muestre la recurrencia que define el costo de la solución óptima

R/ la recurrencia que define el costo de la solución óptima, para nuestro algoritmo es:

Caso base:

lista[0]=0

Relación de recurrencia

$Lista[j] = \max(Lista[j], Lista[j - k] + p * k)$

Donde p es el precio a pagar por el oferente

Donde j es el número actual de acciones $j \in [0, Acciones]$

Donde k son las acciones que se asignan por cada oferente. $k \in [mi, \min(Mi, j)]$

mi es el número mínimo de acciones que se le puede asignar al oferente

Mi es el número máximo de acciones que se le puede asignar al oferente

5. Por medio de un algoritmo, muestre cómo se calcula el costo de la solución óptima a través de subproblemas hasta llegar a dar la solución del problema original

R/

Explicaremos con un ejemplo lo que hace nuestro algoritmo dinámico:

Oferentes: 2

Acciones: 5

Precio mínimo dado por el gobierno: 1

Tomaremos el precio mínimo del gobierno como una oferta:

Oferente 1: precio 1, minimo acciones 0, máximo acciones 5

Oferente 2: precio 3, minimo acciones 1, máximo acciones 2

Oferente 3: precio 2, mínimo acciones 2, máximo acciones 2

Estado inicial

Los números rojos son los índices acciones y la c

Estado de los oferentes según la cantidad de acciones

[oferente1, oferente2, oferente3]

0 [0, 0, 0]

1 [0, 0, 0]

2 [0, 0, 0]

3 [0, 0, 0]

4 [0, 0, 0]

5 [0, 0, 0]

Precio en cada cantidad de acciones

[0, 0, 0, 0, 0, 0]

0 1 2 3 4 5

Luego itera en el primer oferente la cantidad de acciones hasta completar todas las acciones y colocar en la última posición de la lista el valor de asignar las acciones al primer oferente (gobierno)

- Asignamos 5 al oferente 1

[0, 0, 0]

[0, 0, 0]

[0, 0, 0]

[0, 0, 0]

[0, 0, 0]

[5, 0, 0]

Valor de asignar al oferente 1, 5 acciones

[0, 0, 0, 0, 0, 5]

- En la primera iteración de oferentes asignamos todas las acciones al oferente 1, cuando consideramos tener 5, 4 hasta 0 acciones por su respectivo precio: 1

[0, 0, 0]

[1, 0, 0]

[2, 0, 0]

[3, 0, 0]

[4, 0, 0]

[5, 0, 0]

[0, 1, 2, 3, 4, 5]

- Ahora empezamos a iterar en el siguiente oferente las combinaciones de acciones, reutilizando subproblemas anteriores:

Asignamos 1 acción al oferente 2:

[0, 0, 0]

[1, 0, 0]

[2, 0, 0]

[3, 0, 0]

[4, 0, 0]

[4, 1, 0]

[0, 1, 2, 3, 4, 7]

Asignamos 2 al oferente 2 el cual paga 3 por acción:

[0, 0, 0]

[1, 0, 0]

[2, 0, 0]

[3, 0, 0]

[4, 0, 0]

[3, **2**, 0]

[0, 1, 2, 3, 4, **9**]

Solo se puede asignar minimo 1 o maximo 2, el caso en que no se le asigna ya se contempló antes.

En cada iteración de las acciones que se le pueden asignar al oferente 2, validamos si el valor en la posición $\text{valor}[J] < \text{valor}[j - k] + (\text{precio del oferente2} * k)$ donde k es el numero de acciones que estamos contemplando asignarle a este oferente. Si de 5 ($j=5$) acciones, le asigno 2 acciones, es igual a el subproblema donde al oferente 1 le había asignado 3; y a esto sumarle el resultado de multiplicar el precio que paga el oferente 2 por las acciones que le vamos a asignar en esta iteración.

- repetimos el proceso hasta terminar dicho oferente
asignacion:

[0, 0, 0]

[0, 1, 0]

[0, 2, 0]

[1, 2, 0]

[2, 2, 0]

[3, 2, 0]

Valor:

[0, 3, 6, 7, 8, 9]

-luego con el siguiente oferente

[0, 0, 0]

[0, 1, 0]

[0, 2, 0]

[1, 2, 0]

[0, 2, 2]

[1, 2, 2]

[0, 3, 6, 7, 10, **11**]

Dando como resultado que el valor maximo posible de vender las acciones entre las distintas ofertas es **11**, asignando **1** al gobierno, **2** al oferente 2, y **2** al oferente 3

6. Complejidad de las 3 soluciones

n es el número de ofertas.

A es el total de acciones posibles.

R es el rango de asignación de cada oferta.

1. Estrategia por fuerza bruta (solución ingenua)

El tiempo computacional es de $O(n \log(n) + R^n)$ Donde $O(n \log n)$ es por el ordenamiento y $O(R^n)$ es por la generación de todas las combinaciones.

El espacio computacional es de $O(n)$

2. Estrategia de programación dinámica

El tiempo computacional es de $O(n * A * R)$, Esto ocurre, ya que cada combinación de oferta, cantidad de acciones y asignación debe calcularse para optimizar la subasta.

El espacio computacional es de $O(n \times A)$

3. Estrategia de programación voraz

El tiempo computacional es de $O(n(\log(n)))$, Esto ocurre gracias al proceso de ordenamiento de la lista de ofertas, y no a los demás pasos del algoritmo.

El espacio computacional es de $O(n)$

Graficas:

https://docs.google.com/spreadsheets/d/1tj-WZd_ojWkELOOX8XWV4mgfD1HdH7u1T7jVktiqGGw/edit?gid=0#gid=0